Cyprus University of Technology

CSE 226
Algorithms and Complexity

Lecture 4

Recurrences (Part 1)

Αναδρομικές Σχέσεις (Μέρος 1)

Herodotos Herodotou

Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, D. Luebke, and M. Christodoulakis

Recurrence equations

- Recall from the previous lecture
  - An algorithm is **recursive** (**αναδρομικός**) when, in order to solve a problem, it calls itself recursively one or more times
  - When an algorithm contains a recursive call to itself, its running time can often be described by a **recurrence equation** (**αναδρομική εξίσωση**) or simply called a **recurrence**
  - **Recurrence** (**αναδρομή**): an equation that describes a function in terms of its values on smaller inputs
- Example
  - The recurrence equation that describes the worst case running time of Merge-Sort is

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

Methods for solving recurrences

- Three methods!
- The **substitution method** (**μέθοδος αντικατάστασης**)
  - Guess a bound (φράγμα)
  - Use mathematical induction (μαθηματική επαγωγή) to prove it
- The **recursion tree method** (**μέθοδος αναδρομικού δέντρου**)
  - Convert the recurrence into a tree
  - Use techniques for bounding summations (αθροίσματα) to solve the recurrence
- The **master method** (**κεντρική μέθοδος**)
  - Provides bounds for recurrences of the form
    $$T(n) = aT(n/b) + f(n) \quad \text{where } a \geq 1 \text{ and } b > 1$$
  - Once you memorize some cases, it can be easy to find bounds

Common practices & assumptions

- We assume that n is an integer and $n \geq 1$
  - Because it denotes the size of the input to an algorithm
  - Hence, the recurrence of Merge-Sort should be written
    $$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n) & \text{if } n > 1 \end{cases}$$
- Usually, floors and ceilings can be ignored
- For convenience, boundary conditions are usually omitted
  - For sufficiently small n, $\mathrm{T}(n)$ can be considered constant ($\Theta(1)$)
- Example: the Merge-Sort recurrence can be written as
  $$T(n) = 2T(n/2) + \Theta(n)$$

The substitution method

- Two steps
  1. Guess the form of the solution
  2. Use **mathematical induction** (**μαθηματική επαγωγή**) to find the constants and show that the solution works
- Obviously, we can only apply this method in cases when it is easy to guess the form of the answer!
- Can be used to find either **upper or lower bounds** (**άνω ή κάτω φράγματα**)

Example

- Find an upper bound on recurrence:

$$T(n) = 2T(\lfloor n/2 \rfloor) + n$$

- Guess: the solution is $T(n) = O(n \lg n)$
- We will prove this by showing that $T(n) \leq c\, n \lg n$ for some $c > 0$, using mathematical induction
  - **Base case** (**βασικό βήμα**): prove it is true at boundary conditions
  - **Inductive hypothesis** (**επαγωγική υπόθεση**): assume that the bound holds for all $m < n$
  - **Inductive step** (**επαγωγικό βήμα**): show that the bound holds for n

Example (cont'd)

- Recurrence: $T(n) = 2T(\lfloor n/2 \rfloor) + n$
- Need to show: $T(n) \leq cn \lg n$ for some $c > 0$
- Base case: prove it is true at boundary conditions
- Inductive hypothesis: assume bound holds for $\mathrm{m} = \lfloor n/2 \rfloor$
  - That is, assume that $T(\lfloor n/2 \rfloor) \leq c\lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)$
- Inductive step: by substitution we get

$$\begin{aligned} T(n) &\leq 2(c\,\lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \\ &\leq cn \lg(n/2) + n \\ &= cn \lg n - cn \lg 2 + n \\ &= cn \lg n - cn + n \\ &\leq cn \lg n\,, \text{ as long as } -cn + n \leq 0 \Rightarrow c \geq 1 \end{aligned}$$

Example (cont'd)

- Recurrence: $T(n) = 2T(\lfloor n/2 \rfloor) + n$
- Need to show: $T(n) \leq cn \lg n$ for some $c > 0$
- Base case: prove it is true at boundary conditions
  - For $n = 1$ the bound does not hold for any c!
    - $T(1) = 1 \leq c1 \lg 1 = 0 \leftarrow$ This is not possible for any c
  - In asymptotic notation we need to prove $T(n) \leq cn \lg n$ only for values of $n \geq n_0$, for some n_0 of our choosing
  - Choose $n_0 = 2$, replace $T(1) = 1$ with $T(2) = 4$ and $T(3) = 5$
    - $T(2) = 2T(1) + 2 = 4 \leq c\,2 \lg 2$ true for all $c \geq 2$
    - $T(3) = 2T(1) + 3 = 5 \leq c\,3 \lg 3$ true for all $c \geq 2$
  - **Attention!** The base case of the recurrence ($n = 1$) is distinct from the base case of the inductive proof ($n = 2$ and $n = 3$)!

Making a good guess

- Unfortunately, it's no easy task
  - Experience helps!
  - Recursion trees can be useful for making a guess
- If a recurrence looks similar to one we've seen before, it's reasonable to assume a similar solution
  - Example: $T(n) = 2T(\lfloor n/2 \rfloor + 17) + n$
  - It looks quite similar to the one we saw earlier
  - "17" cannot effectively alter the solution to the recurrence, i.e., for large n, the difference between $T(\lfloor n/2 \rfloor)$ and $T(\lfloor n/2 \rfloor + 17)$ is not that large
  - So, we guess $T(n) = O(n \lg n)$ and we prove it with the substitution method

Weak induction assumptions

- Sometimes we make a correct guess but the math doesn't work out in the induction
- Usually, the problem is that the induction assumption isn't strong enough
- Example: $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$
  - We guess it is $O(n)$, and try to show that $T(n) \leq cn$

    $$T(n) \leq c\lfloor n/2 \rfloor + c\lceil n/2 \rceil + 1 = cn + 1$$
  - which does not imply $T(n) \leq cn$ for any choice of c!
  - Notice, we're only off by the constant 1 (but still it's not correct)
  - **Idea!** Subtract a lower-order term from the previous guess

Weak induction assumptions (cont'd)

- Example: $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$
  - So our new guess is $T(n) \leq cn - \mathrm{b}$ where $b \geq 0$ is a constant
  - We have $T(n) \leq (c\lfloor n/2 \rfloor - b) + (c\lceil n/2 \rceil - b) + 1$
    $= cn - 2b + 1$
    $\leq cn - b$
    which is true as long as $-\mathrm{b} + 1 \leq 0 \Rightarrow b \geq 1$
- **Careful!** In induction we need to prove the exact form
- For $T(n) = 2T(\lfloor n/2 \rfloor) + n$ and a guess of $T(n) = O(n)$, it would be wrong to say: $T(n) \leq 2(c\lfloor n/2 \rfloor) + n$
  $\leq cn + n$
  $= O(n)$ ⟸ **Wrong!**

Exercises

1. Show that the solution of
   $$T(n) = T(\lceil n/2 \rceil) + 1$$
   is $\mathrm{O}(\lg n)$.
2. We saw that the solution of
   $$T(n) = 2T(\lfloor n/2 \rfloor) + n$$
   is $\mathrm{O}(n \lg n)$. Show that the solution of this recurrence is also $\Omega(n \lg n)$. Conclude that the solution is $\Theta(n \lg n)$.
3. Show that the solution of
   $$T(n) = 8T(n/2) + n^2$$
   is $\mathrm{O}(n^3)$.

Solutions

Recurrence: $T(n) = T(\lceil n/2 \rceil) + 1$ Show: $T(n) = O(\lg n)$

- Need to show: $T(n) \leq c \lg n$ for some $c > 0$
- Base case:
  - $T(1) = 1 \leq c \lg 1 = 0 \leftarrow$ Not possible
  - Replace $T(1) = 1$ with $T(2) = 2$
  - Then, $T(2) = 2 \leq c \lg 2 = c$, true for all $c \geq 2$
- Inductive hypothesis:
  - Assume that $T(\lceil n/2 \rceil) \leq c \lg(\lceil n/2 \rceil)$
- Inductive step:

$$\begin{aligned} T(n) &\leq c \lg(\lceil n/2 \rceil) + 1 \\ &= c \lg n - c \lg 2 + 1 \\ &= c \lg n - c + 1 \\ &\leq c \lg n, \text{ as long as } -c + 1 \leq 0 \Rightarrow c \geq 1 \end{aligned}$$

Solutions

Recurrence: $T(n) = 2T(\lfloor n/2 \rfloor) + n$ Show: $T(n) = \Omega(n \lg n)$

- Need to show: $T(n) \geq c\, n \lg n$ for some $c > 0$
- Base case:
  - $T(1) = 1 \geq c\, 1 \lg 1 = 0$, true for all $c > 0$
- Inductive hypothesis:
  - Assume that $T(\lfloor n/2 \rfloor) \geq c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)$
- Inductive step:

$$\begin{aligned} T(n) &\geq 2\, c\, \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor) + n \\ &= c\, n \lg n - c\, n \lg 2 + n \\ &= c\, n \lg n - cn + n \\ &\geq c\, n \lg n, \text{ as long as } -cn + n \geq 0 \Rightarrow 0 < c \leq 1 \end{aligned}$$

Solutions

Recurrence: $T(n) = 8T(n/2) + n^2$ Show: $T(n) = O(n^3)$

- Need to show: $T(n) \leq cn^3$ for some $c > 0$
- Base case:
  - $T(1) = 1 \leq c\ 1^3 = c$, true for all $c > 1$
- Inductive hypothesis:
  - Assume that $T(n/2) \leq c(n/2)^3$
- Inductive step:

$$T(n) \leq 8c(n/2)^3 + n^2$$
$$= c\ n^3 + n^2$$

which does not imply $T(n) \leq cn^3$ for any choice of c!

Solutions

Recurrence: $T(n) = 8T(n/2) + n^2$ Show: $T(n) = O(n^3)$

- Need to show: $T(n) \leq cn^3 - dn^2$ for some $c > 0$
- Base case:
  - $T(1) = 1 \leq c\ 1^3 - d\ 1^2 = c - d$, true for all $c > 1 + d$
- Inductive hypothesis:
  - Assume that $T(n/2) \leq c(n/2)^3 - d(n/2)^2$
- Inductive step:

$$T(n) \leq 8c(n/2)^3 - 8d(n/2)^2 + n^2$$
$$= c\ n^3 - 2dn^2 + n^2$$
$$= c\ n^3 - dn^2 - \boldsymbol{dn^2} + \boldsymbol{n^2}$$
$$\leq c\ n^3 - dn^2, \text{ as long as} - \boldsymbol{dn^2} + \boldsymbol{n^2} \leq 0 \Rightarrow d \geq 1$$